
Project Title:

SkyCompare: An Intelligent Flight Comparison Tool

1. Purpose

SkyCompare was built to solve a real problem that every Indian traveler faces - comparing domestic flights is frustrating, slow, and spread across too many websites. The purpose of this project is to build a single, clean, React-only web application that brings flight prices, airline comparisons, baggage policies, on-time performance data, and price intelligence all into one place - without needing a backend server or any external API.

This project demonstrates the complete front-end engineering lifecycle: from component design and state management to routing, form validation, and deployment - covering all 6 Course Outcomes defined in the syllabus.

2. Problem Statement

When a traveler wants to book a domestic flight in India today, they face three specific problems:

Problem 1 — Fragmented Information. To compare IndiGo, Air India, SpiceJet, Vistara, and Go First on the same route, they must open five different websites or use cluttered aggregators that show ads, not answers.

Problem 2 — No Price Intelligence. Existing tools show today's price but never explain whether that price is a good deal, whether it will rise next weekend, or how many seats are left. A traveler booking a flight to Delhi this weekend has no idea that prices jump 40% on Friday.

Problem 3 — Hidden Policies. Baggage allowances, extra baggage fees, on-time performance percentages, and seat availability are buried deep inside airline websites with no way to compare them side by side.

SkyCompare addresses all three with a React SPA that is fast, clean, and requires no backend — all data is deterministically generated from a single `mockData.js` file, making it completely self-contained.

3. Goals

The project had six clear goals, each mapped to a Course Outcome:

Goal 1 — Build a component-driven React architecture with unidirectional data flow and proper separation of concerns. (CO1)

Goal 2 — Use modern JavaScript (ES6+) including arrow functions, destructuring, spread operators, closures via useMemo, and modular file organisation. (CO2)

Goal 3 — Design reusable React components — FlightCard, FilterPanel, CompareTray, SearchBar — each with clear props contracts and local state. (CO3)

Goal 4 — Implement global state management using two Context providers: AppContext for flight data and filters, and AuthContext for user authentication. (CO4)

Goal 5 — Build a complete routing system and form validation using React Router v6 with 7 routes, a protected auth gate, and controlled forms with 4 validation rules. (CO5)

Goal 6 — Use a proper build system and deploy the project using Create React App's Webpack pipeline, with .gitignore, environment variables, and GitHub deployment. (CO6)

4. Scope

In Scope

- Flight search and comparison across 4 routes and 5 airlines
- 10 required modules: Date Price Calendar, Airline Compare, Baggage Policy Grid, OTP Stats, Route Map, Filter Panel, Share Link, Sort Options, Mobile View, Export
- 5 unique intelligence features: Price Predictor, Deal Score, Fare Alert, Seat Availability Trend, Smart Booking Tips
- Complete authentication system: Email/Password login, account creation, simulated Google OAuth, Guest mode
- Animated Welcome Screen post-login
- User avatar and logout in the Navbar
- CSV export and Print/PDF export
- Responsive mobile layout using Flexbox
- Deployment to GitHub

Out of Scope

- Real flight API integration (Skyscanner, Amadeus, MakeMyTrip)
 - Backend server or database
 - Payment gateway or actual booking
 - Real-time seat or price data
 - Push notifications for fare alerts
 - Jest unit tests (future scope)
 - TypeScript migration (future scope)
-

5. Users

SkyCompare is designed for three types of users:

Primary User — The Domestic Traveler. An Indian traveler planning a trip between major cities (Hyderabad, Delhi, Mumbai, Bangalore, Chennai) who wants to compare flights quickly and make a smart booking decision without visiting multiple websites.

Secondary User — The Budget Planner. Someone who is flexible on travel dates and wants to use the Price Calendar and Price Predictor to find the cheapest week to travel.

Tertiary User — The College Examiner / Developer. An academic reviewer or developer who wants to evaluate the application's React architecture, component design, state management, and code quality.

6. Functional Requirements

	Requirement	Implemented In
FR1	User can log in with email/password, Google, or as Guest	<code>AuthPage.js</code>
FR2	User can create a new account with validation	<code>AuthPage.js</code>
FR3	Authenticated user sees Welcome Screen before app	<code>WelcomePage.js</code>
FR4	User can search flights by route and date	<code>SearchBar.js</code>
FR5	User can filter flights by stops and airline	<code>FilterPanel.js</code>

FR6	User can sort flights by price, duration, departure, OTP	<code>AppContext.js</code>
FR7	User can view a 14-day price calendar with trend chart	<code>CalendarPage.js</code>
FR8	User can add up to 3 flights to a compare tray	<code>CompareTrays.js</code>
FR9	User can view side-by-side comparison of flights	<code>ComparePage.js</code>
FR10	User can view baggage policies for all 5 airlines	<code>BaggagePage.js</code>
FR11	User can view OTP stats per airline and per route	<code>OTPPage.js</code>
FR12	User can view an SVG route map of India	<code>MapPage.js</code>
FR13	User can share a link with current search state	<code>AppContext.js</code>
FR14	User can export results as CSV or Print as PDF	<code>SearchBar.js</code>
FR15	User can view 8-week price prediction with recommendation	<code>InsightsPage.js</code>
FR16	User can see a Deal Score (0–100) per flight	<code>InsightsPage.js</code>
FR17	User can set a fare alert with email and target price	<code>InsightsPage.js</code>
FR18	User can view seat availability trend and price surge chart	<code>InsightsPage.js</code>
FR19	User can read smart booking tips	<code>InsightsPage.js</code>
FR20	User can log out from the Navbar	<code>Navbar.js</code>

7. Non-Functional Requirements

Requirement

How it's met

NFR1	Performance — App must load and render fast	<code>useMemo</code> and <code>useCallback</code> prevent unnecessary re-renders. Mock data avoids API latency.
NFR2	Responsiveness — Must work on mobile and desktop	Flexbox <code>flexWrap</code> layout on all pages. Grid with <code>auto-fill</code> and <code>minmax</code> in <code>CalendarPage</code> and <code>OTPPage</code> .
NFR3	Maintainability — Code must be clean and modular	19 files separated into <code>context/</code> , <code>data/</code> , <code>components/</code> , <code>pages/</code> . Each file has one responsibility.
NFR4	Accessibility — Must follow basic ARIA standards	ARIA <code>role</code> and <code>aria-label</code> on SVG Route Map. Semantic HTML in forms.
NFR5	Security — Auth state must be protected	<code>AuthGate</code> in <code>App.js</code> prevents access to the main app if <code>user === null</code> .
NFR6	Deployability — Must run on a custom port	<code>PORT=3001 npm start</code> environment variable used to avoid conflicts.
NFR7	Version Control — Must be tracked in Git	<code>.gitignore</code> excludes <code>node_modules</code> . Full project pushed to GitHub.

8. Assumptions

1. All flight data is mock-generated deterministically using `mockData.js`. No real API is called.
2. The "Google Login" button simulates OAuth — it does not connect to Google's actual authentication servers.
3. The "Fare Alert" email input simulates setting an alert — no actual email is sent.
4. "Export CSV" uses the browser's native `Blob` and `URL.createObjectURL` API — no server is involved.
5. "Share Link" uses `navigator.clipboard.writeText()` — it copies a URL to clipboard but does not use a URL shortener or backend.
6. All prices are deterministic random values generated from seed formulas — they are consistent per route/date combination but are not real fares.
7. The application is designed for modern browsers (Chrome, Safari, Firefox) and does not support Internet Explorer.
8. The user's session is in-memory only — refreshing the browser will log the user out, as there is no `localStorage` or session persistence.

9. New accounts created during signup are stored in a local React state array and are lost on page refresh.
-

9. Acceptance Criteria

Acceptance Criterion

- AC1 All 10 required modules from Problem 22 are implemented and functional
- AC2 All 6 Course Outcomes are demonstrably covered by specific features and files
- AC3 Login with email/password, Google, and Guest mode all work without errors
- AC4 New account creation with all 4 validation rules (empty, length, match, duplicate) works
- AC5 Welcome Screen appears after login and auto-redirects in 3 seconds
- AC6 Flights are correctly filtered by stops and airline
- AC7 Flights are correctly sorted by all 4 sort options
- AC8 Price Calendar displays 14 days and clicking a date updates flights
- AC9 Compare Tray accepts up to 3 flights and navigates to Compare page
- AC10 CSV export downloads a valid `.csv` file with all flight data
- AC11 Price Predictor shows 8-week chart with a Book Now / Wait / Flexible recommendation
- AC12 Fare Alert validates email input and shows confirmation state

AC13 Application is publicly accessible on GitHub

AC14 Application runs on `PORT=3001` without conflict

AC15 Layout is responsive and usable on mobile screen widths